Project Documentation

Author

Full Name: Jeevan Jyoti
Roll Number: D23F1000703
Student Email: 23f1000703@ds.study.iitm.ac.in
About Me: I am a student of IITM Madras BS Degree Course.

Description

This project is a car parking application where users can easily book a parking spot, and the admin can view, add, or delete spots, along with details of each spot.
AI/LLM Used: 36% — Help was taken primarily for HTML and CSS components to improve the user interface.

Technologies Used

- Flask: Lightweight web framework used for building the backend of the application.
- Flask-SQLAlchemy: ORM library used to manage and interact with the database efficiently.
- SQLite: A serverless, self-contained database engine used for storing user and parking data.
- Jinja2: Used for rendering HTML templates dynamically with data from the backend.
- Bootstrap: Used for building a responsive and mobile-friendly frontend design.
- HTML/CSS: Core technologies for creating and styling the user interface.
- Flask-Login: Used to manage user sessions and authentication securely.
- datetime, matplotlib, os: Used for date/time handling, chart generation, and system-level operations.

Database Schema Design

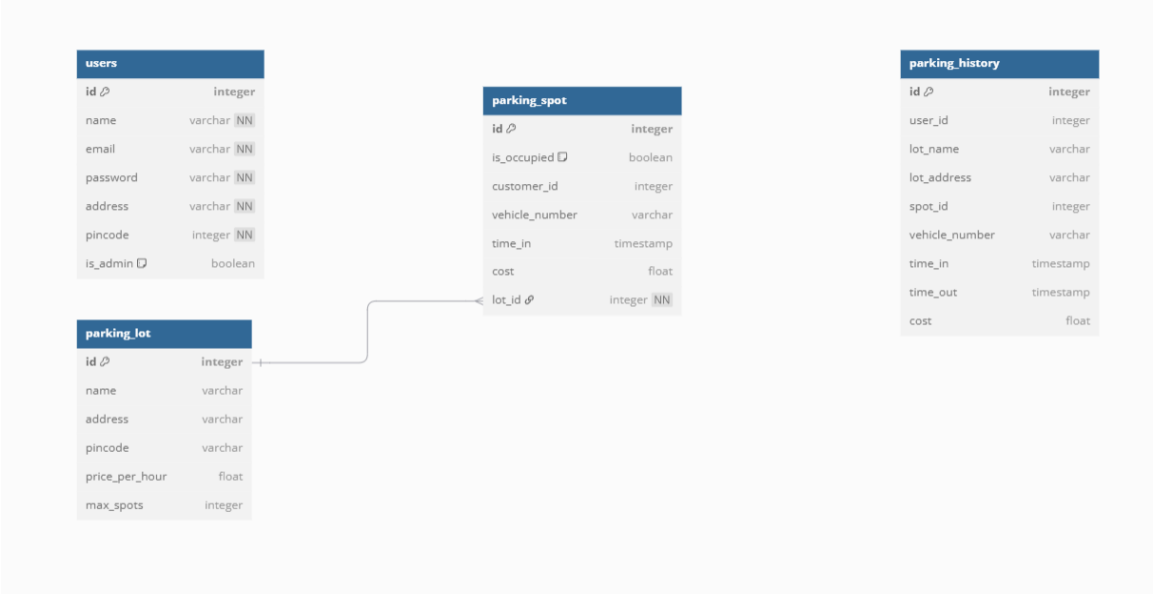
1. User
- id (Integer, Primary Key, Auto Increment)
- name (String, Not Null)
- email (String, Unique, Not Null)
- password (String, Not Null)
- address (String, Not Null)
- pincode (Integer, Not Null)
- is_admin (Boolean, Default: False)

2. ParkingLot
- id (Integer, Primary Key)
- name (String)
- address (String)

- pincode (String)
- price_per_hour (Float)
- max_spots (Integer)
- spots (Relationship with ParkingSpot, Cascade Delete)

3. ParkingSpot
- id (Integer, Primary Key)
- is_occupied (Boolean, Default: False)
- customer_id (Integer, Nullable)
- vehicle_number (String, Nullable)
- time_in (DateTime, Nullable)
- cost (Float, Nullable)
- lot_id (Foreign Key → ParkingLot.id)

4. ParkingHistory
- id (Integer, Primary Key)
- user_id (Integer)
- lot_name (String)
- lot_address (String)
- spot_id (Integer)
- vehicle_number (String)
- time_in (String)
- time_out (String)
- cost (Float)



API Design

This project does not implement dedicated REST APIs or YAML-based API definitions. All functionalities are handled using traditional Flask routes and form submissions via the frontend.

Architecture and Features

The project is organized into four main folders:

- controllers/: Contains all the Flask routes and business logic that handle requests and interact with the database.
- instance/: Holds the SQLite database and configuration files for the app.
- static/: Includes all static assets such as CSS, JavaScript, and images used in the frontend.
- templates/: Contains all HTML files built using Jinja2 for dynamic content rendering.

Key Features Implemented:

- User Registration and Login – Managed using Flask-Login for both users and admin roles.
- Admin Dashboard – Admins can add, view, and delete parking lots and spots.
- User Dashboard – Users can search and book available parking spots.
- Real-time Parking Spot Status – Admins can see which spots are occupied and by whom.
- Parking History Tracking – Users' parking sessions are stored and viewable in a history log.
- Responsive UI – Designed using Bootstrap and enhanced with HTML/CSS for ease of use.
- Charts/Reports – Implemented using matplotlib to generate visual statistics (if used).

Video

Link of the video report is

https://drive.google.com/file/d/1ILXWyrql26zfjqm_dOwLR7dr-j4HW4o7/view?usp=sharing